# 1η ΑΣΚΗΣΗ ΣΤΗΝ ΑΡΧΙΤΕΚΤΟΝΙΚΗ ΥΠΟΛΟΓΙΣΤΩΝ

Όνομα: Δημήτριος

Επώνυμο: Μιχαλακόπουλος

ΑΜ: 03120229

## ΜΕΡΟΣ Α

```
# Register Mapping:
# $a0 = int* a (base address of array)
# $a1 = int length
# $s1 = min (must be saved to stack because it's a saved register)
# $t0 = i (loop counter)
# $t2 = p (pointer address)
# $s2 = *p (current value loaded from array)

findmin:
    # --- PROLOGUE: Save registers to stack ---
    addi $sp, $sp, -8      # Adjust stack pointer to make room for 2 items
    sw   $ra, 4($sp)       # Save return address
    sw   $s1, 0($sp)       # Save $s1 (callee-saved) so we can use it for 'min'

    # --- INITIALIZATION ---
    addi $t0, $zero, 1     # i = 1
    addi $t2, $a0, 0       # p = &a[0] (copy base address from argument $a0)
    lw   $s1, 0($t2)       # min = *p (load the 1st element into min)

LOOP:
    # --- LOOP CONDITION ---
    beq  $t0, $a1, END     # if (i == length) goto END

    # --- POINTER ARITHMETIC ---
    addi $t2, $t2, 4       # p++ (increment pointer by 4 bytes for next int)
    lw   $s2, 0($t2)       # Load current value (*p) into temporary register $s2

    # --- MIN CHECK: if (*p < min) ---
    slt  $t1, $s2, $s1     # Set $t1 = 1 if (*p < min), else $t1 = 0
    beq  $t1, $zero, NEXT  # If $t1 is 0 (false), skip the update
    add  $s1, $s2, $zero   # min = *p (update min with the new smaller value)

NEXT:
    # --- LOOP INCREMENT ---
    addi $t0, $t0, 1       # i++
    j    LOOP              # Jump back to start of loop

END:
```

```
    # --- EPILOGUE: Restore and Return ---
    add  $v0, $s1, $zero   # Place 'min' into return register $v0
    lw   $s1, 0($sp)       # Restore original $s1 value
    lw   $ra, 4($sp)       # Restore return address
    addi $sp, $sp, 8       # Restore stack pointer
    jr   $ra               # Return to caller
```

**ΜΕΡΟΣ Β**

```
# Register Mapping:
# $a0 = int* w (base address of weights array)
# $a1 = int* x (base address of inputs array)
# $a2 = int n (length of arrays)
# $a3 = int bias (bias value)
# $t0 = i (loop counter)
# $v0 = sum (accumulator, initialized to bias)
# $t3 = w[i] (current weight value)
# $t5 = x[i] (current input value)

neuron:
    # --- INITIALIZATION ---
    li   $t0, 0           # i = 0
    move $v0, $a3         # sum = bias (initialize accumulator with bias argument)

LOOP:
    # --- LOOP CONDITION ---
    bge  $t0, $a2, END    # if (i >= n) goto END

    # --- ARRAY ADDRESSING ---
    sll  $t1, $t0, 2      # offset = i * 4 (convert index to byte offset)

    # Load w[i]
    add  $t2, $a0, $t1    # p1 = &w[i] (base w + offset)
    lw   $t3, 0($t2)      # load w[i] into $t3

    # Load x[i]
    add  $t4, $a1, $t1    # p2 = &x[i] (base x + offset)
    lw   $t5, 0($t4)      # load x[i] into $t5

    # --- ACCUMULATION ---
    mul  $t6, $t3, $t5    # product = w[i] * x[i]
    add  $v0, $v0, $t6    # sum = sum + product

    # --- LOOP INCREMENT ---
    addi $t0, $t0, 1      # i++
    j    LOOP             # Jump back to start of loop

END:
    # --- ACTIVATION FUNCTION (ReLU) ---
    bgez $v0, SKIP        # if (sum >= 0) goto SKIP
```

```
    move $v0, $zero       # sum = 0 (if sum < 0, clamp it to 0)

SKIP:
    # --- EPILOGUE: Return ---
    jr   $ra              # Return sum (value in $v0)
```

**ΜΕΡΟΣ Γ**

```
# Register Mapping:
# $a0 = int num_layers (argument)
# $s0 = int* input (pointer to current layer input in X)
# $s1 = int* wptr (pointer to weights W)
# $s2 = int* bptr (pointer to biases B)
# $s3 = int layer (outer loop counter)
# $s4 = int j (inner loop counter)
# $s5 = int num_layers (saved copy of argument)
# $s6 = int n_in (number of inputs for current layer)
# $s7 = int n_out (number of neurons in current layer)

network:
    # --- PROLOGUE: Save registers to stack ---
    addi $sp, $sp, -36        # Adjust stack pointer (allocate 36 bytes for 9 items)
    sw   $ra, 32($sp)         # Save return address
    sw   $s0, 28($sp)         # Save $s0 (input pointer)
    sw   $s1, 24($sp)         # Save $s1 (wptr)
    sw   $s2, 20($sp)         # Save $s2 (bptr)
    sw   $s3, 16($sp)         # Save $s3 (layer)
    sw   $s4, 12($sp)         # Save $s4 (j)
    sw   $s5, 8($sp)          # Save $s5 (num_layers)
    sw   $s6, 4($sp)          # Save $s6 (n_in)
    sw   $s7, 0($sp)          # Save $s7 (n_out)

    # --- INITIALIZATION ---

    # int *input = X;
    la   $s0, X               # $s0 = &X[0] (input pointer)

    # int *wptr = W;
    la   $s1, W               # $s1 = &W[0] (weight pointer)

    # int *bptr = B;
    la   $s2, B               # $s2 = &B[0] (bias pointer)

    move $s5, $a0             # Save num_layers argument into $s5

    # --- OUTER LOOP: for (layer = 1; layer < num_layers; layer++) ---
    li   $s3, 1               # layer = 1

OUTER_LOOP:
    # --- OUTER LOOP CONDITION ---
```

```
    bge  $s3, $s5, EXIT_NETWORK # if (layer >= num_layers) goto EXIT_NETWORK

    # --- LOAD LAYER SIZES FROM LAYERS[] ---
    la   $t0, LAYERS          # Load base address of LAYERS array

    # Get n_in = LAYERS[layer-1]
    sub  $t1, $s3, 1          # $t1 = layer - 1
    sll  $t1, $t1, 2          # Convert index to bytes (* 4)
    add  $t1, $t0, $t1        # Address of LAYERS[layer-1]
    lw   $s6, 0($t1)          # Load n_in into $s6

    # Get n_out = LAYERS[layer]
    sll  $t2, $s3, 2          # Convert layer index to bytes (* 4)
    add  $t2, $t0, $t2        # Address of LAYERS[layer]
    lw   $s7, 0($t2)          # Load n_out into $s7

    # --- INNER LOOP: for (j = 0; j < n_out; j++) ---
    li   $s4, 0               # j = 0

INNER_LOOP:
    # --- INNER LOOP CONDITION ---
    bge  $s4, $s7, END_INNER  # if (j >= n_out) goto END_INNER

    # --- PREPARE ARGUMENTS FOR neuron() ---
    # Call: neuron(wptr, input, n_in, *bptr)
    # Mapping for neuron: $a0=weights, $a1=inputs, $a2=n, $a3=bias

    # 1. $a0 = wptr (Stored in $s1)
    move $a0, $s1             # Copy current weight pointer

    # 2. $a1 = input (Stored in $s0)
    move $a1, $s0             # Pass the current input pointer directly

    # 3. $a2 = n_in (Stored in $s6)
    move $a2, $s6             # Copy n_in

    # 4. $a3 = *bptr (Value pointed to by $s2)
    lw   $a3, 0($s2)          # Load the bias value from memory

    # --- CALL FUNCTION ---
    jal  neuron               # Jump and Link to neuron (result in $v0)

    # --- STORE RESULT ---
    # target address = input + (n_in + j) * 4
    # We write to the next layer's section in X (relative to current position)

    add  $t4, $s6, $s4        # $t4 = n_in + j
    sll  $t4, $t4, 2          # $t4 = (n_in + j) * 4 (bytes)
    add  $t5, $s0, $t4        # $t5 = input pointer + offset_bytes
    sw   $v0, 0($t5)          # Store result in X array
```

```
    # --- UPDATE POINTERS ---
    # wptr += n_in
    sll  $t6, $s6, 2          # $t6 = n_in * 4 (bytes)
    add  $s1, $s1, $t6        # Advance wptr ($s1)

    # bptr += 1
    addi $s2, $s2, 4          # Advance bptr ($s2) by 4 bytes

    # --- INNER LOOP INCREMENT ---
    addi $s4, $s4, 1          # j++
    j    INNER_LOOP           # Jump to start of inner loop

END_INNER:
    # --- UPDATE INPUT POINTER (OFFSET) ---
    # input += n_in
    # Advance the input pointer to the start of the next layer
    sll  $t8, $s6, 2          # $t8 = n_in * 4 (bytes)
    add  $s0, $s0, $t8        # Advance the input pointer ($s0)

    # --- OUTER LOOP INCREMENT ---
    addi $s3, $s3, 1          # layer++
    j    OUTER_LOOP           # Jump to start of outer loop

EXIT_NETWORK:
    # --- RETURN VALUE ---
    # return *input
    # At the end, $s0 points to the final output in X
    lw   $v0, 0($s0)          # Load final result into return register $v0

    # --- EPILOGUE: Restore and Return ---
    lw   $s7, 0($sp)          # Restore $s7
    lw   $s6, 4($sp)          # Restore $s6
    lw   $s5, 8($sp)          # Restore $s5
    lw   $s4, 12($sp)         # Restore $s4
    lw   $s3, 16($sp)         # Restore $s3
    lw   $s2, 20($sp)         # Restore $s2
    lw   $s1, 24($sp)         # Restore $s1
    lw   $s0, 28($sp)         # Restore $s0
    lw   $ra, 32($sp)         # Restore return address
    addi $sp, $sp, 36         # Restore stack pointer
    jr   $ra                  # Return to caller
```

## ΜΕΡΟΣ Δ

```
.data
    # ============================================================================
    # DATA SECTION
    # ============================================================================
```

```
    # LAYERS: Defines structure (Input=3, Hidden=2, Output=1)
    LAYERS: .word 3, 2, 1

    # X: Input/Output Array (Total size = 3 + 2 + 1 = 6 integers)
    # Input: [29, 50, 20] because last two digits of AM=YZ=29
    X:      .word 29, 50, 20, 0, 0, 0

    # W: Weights
    # Layer 1 (to h0): -2, 5, 3
    # Layer 1 (to h1): -5, 2, 2
    # Layer 2 (to y):  1, 1
    W:      .word -2, 5, 3, -5, 2, 2, 1, 1

    # B: Biases
    # h0=0, h1=0, y=0
    B:      .word 0, 0, 0

    # STRINGS
    msg_result: .asciiz "Neural Network Output: "
    msg_class1: .asciiz "\nResult: The digit is 1\n"
    msg_class7: .asciiz "\nResult: The digit is 7\n"

.text
.globl main

# ============================================================================
# MAIN FUNCTION (PART D)
# ============================================================================
main:
    # --- 1. PREPARE ARGUMENTS ---
    # The network function takes num_layers as an argument ($a0)
    li   $a0, 3

    # --- 2. CALL NETWORK ---
    jal  network          # Call Part C function

    # Result is returned in $v0. Save it to $s0 safely.
    move $s0, $v0

    # --- 3. PRINT NUMERICAL RESULT ---
    # Print label
    li   $v0, 4           # syscall: print_string
    la   $a0, msg_result
    syscall

    # Print integer
    li   $v0, 1           # syscall: print_int
    move $a0, $s0
    syscall
```

```
    # --- 4. CLASSIFICATION (Threshold Check) ---
    # Threshold = 230
    # if (output >= 230) print "7" else print "1"

    li   $t0, 230
    bge  $s0, $t0, PRINT_7

PRINT_1:
    li   $v0, 4           # syscall: print_string
    la   $a0, msg_class1
    syscall
    j    EXIT_MAIN

PRINT_7:
    li   $v0, 4           # syscall: print_string
    la   $a0, msg_class7
    syscall

EXIT_MAIN:
    # --- 5. EXIT ---
    li   $v0, 10          # syscall: exit
    syscall

# ================================================================================
# NETWORK FUNCTION (PART C)
# ================================================================================
# Register Mapping:
# $a0 = int num_layers (argument, saved to stack at 0($sp))
# $s0 = int* input (pointer to current layer input in X)
# $s1 = int* wptr (pointer to weights W)
# $s2 = int* bptr (pointer to biases B)
# $s3 = int layer (outer loop counter)
# $s4 = int j (inner loop counter)
# $s5 = int n_in (inputs per neuron in current layer)
# $s6 = int n_out (neurons in current layer)

network:
    # --- PROLOGUE: Save registers to stack ---
    addi $sp, $sp, -36       # Allocate 36 bytes
    sw   $ra, 32($sp)        # Save return address
    sw   $s0, 28($sp)        # Save $s0
    sw   $s1, 24($sp)        # Save $s1
    sw   $s2, 20($sp)        # Save $s2
    sw   $s3, 16($sp)        # Save $s3
    sw   $s4, 12($sp)        # Save $s4
    sw   $s5, 8($sp)         # Save $s5
    sw   $s6, 4($sp)         # Save $s6
    sw   $a0, 0($sp)         # Save num_layers ($a0) to stack

    # --- INITIALIZATION ---
```

```
    la   $s0, X              # $s0 = &X[0] (input pointer)
    la   $s1, W              # $s1 = &W[0] (weight pointer)
    la   $s2, B              # $s2 = &B[0] (bias pointer)

    # --- OUTER LOOP: for (layer = 1; layer < num_layers; layer++) ---
    li   $s3, 1              # layer = 1

OUTER_LOOP:
    # Check Loop Condition (using num_layers from stack)
    lw   $t0, 0($sp)         # Load num_layers
    bge  $s3, $t0, EXIT_NETWORK

    # --- LOAD LAYER SIZES FROM LAYERS[] ---
    la   $t0, LAYERS

    # n_in = LAYERS[layer-1]
    sub  $t1, $s3, 1         # layer - 1
    sll  $t1, $t1, 2         # * 4
    add  $t1, $t0, $t1       # address
    lw   $s5, 0($t1)         # $s5 = n_in

    # n_out = LAYERS[layer]
    sll  $t2, $s3, 2         # layer * 4
    add  $t2, $t0, $t2       # address
    lw   $s6, 0($t2)         # $s6 = n_out

    # --- INNER LOOP: for (j = 0; j < n_out; j++) ---
    li   $s4, 0              # j = 0

INNER_LOOP:
    bge  $s4, $s6, END_INNER

    # --- PREPARE ARGUMENTS FOR neuron() ---
    move $a0, $s1            # wptr
    move $a1, $s0            # input pointer
    move $a2, $s5            # n_in
    lw   $a3, 0($s2)         # *bptr (load bias value)

    jal  neuron              # Call neuron

    # --- STORE RESULT ---
    # target = input + (n_in + j)
    add  $t4, $s5, $s4       # n_in + j
    sll  $t4, $t4, 2         # to bytes
    add  $t5, $s0, $t4       # address in X
    sw   $v0, 0($t5)         # Store result

    # --- UPDATE POINTERS ---
    # wptr += n_in
    sll  $t6, $s5, 2         # n_in * 4
```

```
    add  $s1, $s1, $t6

    # bptr += 1
    addi $s2, $s2, 4

    # --- INNER LOOP INCREMENT ---
    addi $s4, $s4, 1          # j++
    j    INNER_LOOP

END_INNER:
    # --- UPDATE INPUT POINTER ---
    # input += n_in
    sll  $t8, $s5, 2          # n_in * 4
    add  $s0, $s0, $t8        # Advance input pointer

    # --- OUTER LOOP INCREMENT ---
    addi $s3, $s3, 1          # layer++
    j    OUTER_LOOP

EXIT_NETWORK:
    # --- RETURN VALUE ---
    lw   $v0, 0($s0)          # return *input

    # --- EPILOGUE ---
    lw   $s6, 4($sp)
    lw   $s5, 8($sp)
    lw   $s4, 12($sp)
    lw   $s3, 16($sp)
    lw   $s2, 20($sp)
    lw   $s1, 24($sp)
    lw   $s0, 28($sp)
    lw   $ra, 32($sp)
    addi $sp, $sp, 36
    jr   $ra

# ============================================================================
# NEURON FUNCTION (PART B)
# ============================================================================
# Register Mapping:
# $a0 = int* w
# $a1 = int* x
# $a2 = int n
# $a3 = int bias
# $v0 = sum

neuron:
    li   $t0, 0           # i = 0
    move $v0, $a3         # sum = bias

NEURON_LOOP:
```

```
    bge  $t0, $a2, NEURON_END

    sll  $t1, $t0, 2      # offset = i * 4

    add  $t2, $a0, $t1    # &w[i]
    lw   $t3, 0($t2)      # w[i]

    add  $t4, $a1, $t1    # &x[i]
    lw   $t5, 0($t4)      # x[i]

    mul  $t6, $t3, $t5    # w[i] * x[i]
    add  $v0, $v0, $t6    # sum += product

    addi $t0, $t0, 1      # i++
    j    NEURON_LOOP

NEURON_END:
    bgez $v0, NEURON_SKIP
    move $v0, $zero       # ReLU: if sum < 0, sum = 0

NEURON_SKIP:
    jr   $ra
```

**ΕΙΣΟΔΟΣ:** [29 50 20], αφού για ΑΜ=03120229 το ΥΖ=29

**ΕΞΟΔΟΣ:** Neural Network Output: 252

Result: The digit is 7